Notes for ECE 50877 - Introduction to Applied Cryptography

Zeke Ulrich

November 3, 2026

Contents

Course Description 2
Basis of Cryptography 3
Computational Security 5
Pseudorandom Number Generators 6
Private Key Encryption 7
Stream Ciphers 9
Block Ciphers 10
Message Authenication Code 11
Authenticated Encryption 12
Hash Functions 13
Practical Constructions 14
- *Stream Ciphers* 14
- *Block Ciphers* 14
- *Hash Functions* 15

Theory 16
- *Modulus* 16
- *Groups* 16

Public Key Cryptography 18
Public Key Encryption 19
Digital Signatures 20
Public Key Infrastructure 22
System and Network Security Applications 24
- *Multi-Factor Authentication* 24
- *Universal Second Factor* 24

Secret Sharing 25

Threshold 26

Threshold Encryption 26

Multi-party Computation 26

Oblivious Transfer 27

Yao's Garbled Circuits 28

Course Description

Cryptography is an essential component of securing communication and data in a wide range of applications. This course explores fundamental building blocks in cryptography such as ciphers, hash functions, and digital signatures. We will then demonstrate how these building blocks are used practically to secure communications and systems. We will also learn about the power of advanced cryptography to enable complex functionalities with rigorous security guarantees. We will cover fundamental secure multiparty computation protocols, homomorphic encryption, and blockchains, and review their application in building privacy-preserving machine learning. We will also explore social and ethical issues in developing and deploying cryptographic systems.

Basis of Cryptography

Consider the archetypical case in cryptography. Alice wants to send a message to Bob, and wants the contents to remain secret from Vincent. Alice encrypts her message with a secret key s.

An encryption scheme (Gen, Enc, Dec) is a triple of algorithms where

- Key generation $\text{Gen}(1^n)$ outputs a secret key k of length n (also called the security parameter).
- Encryption algorithm $\text{Enc}_k(m) = c$ outputs ciphertext c.
- Decryption algorithm $\text{Dec}_k(c) = m$ outputs plaintext m.

An encryption algorithm is defined over a message space M and a key space K. Requirements:

- Correctness: $\text{Dec}_k(\text{Enc}_k(m)) = m$.
- Security: definition with respect to adversaries.

The secret key k should be hidden from the adversary. The encryption and decryption algorithms themselves should be public; this is Kerckhoffs's principle. This is because the secrey keys are easy to hide and replace in case of a breach, not so with algorithms. It is also good to have many eyes on an algorithm to expose security flaws.

The key space must be large enough such that a brute force attack is infeasible. Consider a Caesar cipher; the key space is only 26 long and a message so encrypted is quickly bruteforced.

Shannon's Treatment: messages come from some distribution, let M be a random variable for sampling the messages from the message space M. Distribution M is known to the adversary. This captures a priori information about the messages. The ciphertext $c = \text{Enc}_k(m)$ depends on m chosen according to M. k chosen randomly according to Gen. Enc may also use some randomness. Dec is deterministic. These induce a distribution C over the ciphertexts c. The adversary only observes c. Knowledge about m before observing the output of C is captured by M. Knowledge about m after observing the output of C is captured by $M|C$. An encryption scheme (Gen, Enc, Dec) is *Shannon secure* if for every probability distribution for M, every message $m \in M$, and every ciphertext $c \in C$: $Pr[M = m|C = c] = Pr[M = m]$.

An alternative defition of secrecy is *Perfect Secrecy*. An encryption scheme is perfectly secure for every pair of messages m_1 and $m_2 \in M$ and every ciphertext $c \in C$ if $Pr[\text{Enc}_k(m_1) = c] = Pr[\text{Enc}_k(m_2) = c]$.

Shannon had fingers in many cryptographic pies. Shannon's Theorem says that if (Gen, Enc, Dec) is a perfectly secret encryption scheme with message space M and key space K, then $|K| \geq |M|$. This means

that for an encryption scheme to be perfectly secret, the keys must be as long as the message. The challenge of modern cryptography is to make something just as good as perfect for practical purposes.

Computational Security

A scheme is (t, ε)-secure if any adversary running for time at most t succeeds in breaking the scheme with probability at most ε. However, this definition makes no guarantees for after t; perhaps the probability of an attacker succeeding rises sharply after t.

A private-key encryption scheme with key length n and for an adversary running for time t, succeeds in breaking the scheme with probability at most $c\frac{t}{2^n}$ (for some constant c).

A better approach to security is to say a scheme is secure if any *efficient* adversary succeeds in breaking the scheme with at most *negligible* probability.

A function f (from natural numbers to non-negative real numbers) is polynomially bounded (or polynomial) if there is a constant c such that $f(n) < n^c$ for all n.

An algorithm A runs in polynomial time if there is a polynomial p s.t. for all inputs $x \in \{0,1\}^*$, $A(x)$ terminates within $p(|x|)$ steps.

An algorithm is efficient if it runs in polynomial time.

A function $v(n)$ is negligible if for every c, there exists some N such that for all $n > N$, $v(n) \leq \frac{1}{n^c}$.

A scheme is computationally secure if for every probabilistic polynomial-time (PPT) adversary A carrying out an attack, the probability that A succeeds is negligible.

Pseudorandom Number Generators

Cryptography needs a lot of random to be secure. Computers supply a little bit through core temperature, power consumption, mouse movement, and it would be nice to expand those random bits into many more random bits.

Specifically, given n uniformly random bits $x = x_1|x_2|\dots|x_n$, we want a deterministic polynomial-time algorithm G such that $G(x)$ outputs $l(n)$ bits $y = y_1|y_2|\dots|y_{l(n)}$ with $l(n) > n$ and y looks "as good as" a truly random string. $G : \{0,1\}^n \to \{0,1\}^{l(n)}$ is called a pseudorandom generator (PRG). "As good as" means that the string will pass all statistical tests that a truly random string would pass. You can never generate more information from nothing, so the idea of "as good as" means that no efficient computer can tell $G(x)$ and a truly random string apart. The distributions

$$\left\{x \leftarrow \{0,1\}^{l(n)} : G(x)\right\} \tag{1}$$

and

$$\left\{r \leftarrow \{0,1\}^{l(n)} : r\right\} \tag{2}$$

are "computationally indistinguishable".

"Computationally indistinguishable" means that no PPT distinguisher algorithm D can tell them apart.

A deterministic algorithm G is called a pseudorandom generator if:

- G can be computed in polynomial time.
- $|G(x)| > |x|$
- $\{x \leftarrow \{0,1\}^n : G(x)\} \approx_c \left\{U_{l(n)}\right\}$ where $l(n) = |G(0^n)|$

$\approx_c$ denotes computational indistinguishability. $U_{l(n)}$ denotes uniform distribution over $\{0,1\}^{l(n)}$. $|G(x) = l(n)|$ is the expansion factor of G.

Private Key Encryption

Recall that for OTPs, n bits of key were required. With PRGs, we can create n bits of "good-enough" random out of fewer than n bits. The secret seed for the PRG takes the place of a key.

Consider an efficient eavesdropping adversary observing the ciphertext. The encryption scheme is secure if the adversary can guess better than random with negligible $v(n)$ probability.

We can define security as having indistinguishable encryptions in the presence of an eavesdropper (EAV-secure) thus: a symmetric encryption scheme (Gen, Enc, Dec) is secure if $\forall m_0, m_1 : \{\text{Enc}_k(m_0)\} \approx_c \{\text{Enc}_k(m_1)\}$ for $k \leftarrow^{\$} \{0,1\}^n$.

Consider:

- G is a PRG
- $\text{Gen}(1^n) : k \leftarrow^{\$} \{0,1\}^n$
- $\text{Enc}_k(m) = m \oplus G(k)$
- $\text{Dec}_k(c) = c \oplus G(k)$

We wish to prove PRG encryption is EAV-secure. We may prove by reduction. We want to show construction Π is secure. We want to show that Π is secure as long as some underlying problem X is hard. The idea is to show that if you break Π then you also solve X. By contrapositive, then if X is hard then Π is also hard.

Proofs by reduction follow this template:

- We want to show a construction Π secure
- We want to show that Π is secure as long as some underlying problem X is hard
- An algorithm that "breaks" Π can be turned into an algorithm that "breaks" X.
- Therefore, if breaking Π would break X, then Π is secure as long as X is hard. Basically, if breaking Π would require an attacker to break X, then breaking Π is at least as hard as breaking X

We will study these attack models:

- Ciphertext only attack: eavesdropper observes multiple ciphertexts between Alice and Bob, possibly of the same message, and has prior knowledge of the distribution of messages sent (e.g. like letter frequency in the English language). However, ideally an adversary would not be able to tell if the same message is sent. However, if an algorithm is deterministic, then the same message produces the same ciphertext. We will deal with this later

- Known plaintext attack: adversary can see plaintext-ciphertext pairs
- Chosen plaintext attack: adversary picks what messages it can see the encryptions of
- Chosen ciphertext attack: adversary can gather information by obtaining the decryptions of chosen ciphertexts.

Model A is stronger than model B, if a cipher secure against A is also secure against B. It is a fact that CTO $<$ KPA $<$ CPA $<$ CCA.

We want to build a PRG encryption scheme that is CPA-secure. Consider psuedorandom functions, "random-looking" functions.

A random function f from n bits to n bits is a function selected uniformly at random from all 2^{n2^n} functions that map n bits to n bits.

A psuedorandom function F is indistinguishable from a random uniform function such that every polynomial time distinguisher D that receieves a description of F outputs 1 with "almost" the same probability as when it receieves a description of a random function f.

Let F be a pseudorandom function.

$$\text{Gen}(1^n) : k \leftarrow^{\$} \{0,1\}^n \tag{3}$$

$$\text{Enc}(k, m) : \text{Pick} r \leftarrow^{\$} \{0,1\}^n .\text{Output}(r, m \oplus F_k(r)) \tag{4}$$

$$\text{Dec}(k, (r, c)) : \text{Output} c \oplus F_k(r) \tag{5}$$

The above construction is a CPA-secure, fixed-length private-key encryption scheme for messages of length n.

Stream Ciphers

We have seen thus far how to build EAV-secure and CPA-secure encryptions. However, there are some practical issues, such as that they are only defined for fixed-length inputs/outputs.

A stream cipher generates output bits on demand. A stream cipher is a pair of deterministic algorithms (`init`, `next`). `init` takes input seed s and an initialization vector IV and outputs initial state st. `next` takes current state st and outputs a bit y and updated state st'. A stream cipher can be constructed from a pseudorandom function.

Block Ciphers

We use block ciphers to encrypt long (variable input length) messages. We define a block cipher as a pseudorandom permutation (PRP). PRP is a pseudorandom function with the constraint that the entries in any two distinct rows (in a function look-up table) must be different. A PRP is indistinguishable from a random permutation. $2^n!$ random permutations exist for n bits.

Block ciphers have different modes:

- Electronic Code Book (ECB)
- Cipher Block Chaining (CBC)
- Output Feedback (OFB)
- Counter (CTR)

If we use the electronic code book (ECB) mode of operation where we chunk the plaintext and apply the block cipher to each plaintext block, this isn't even EAV-secure since it's deterministic. If a block is repeated in plaintext, it will result in a repeating block in the ciphertext. This could leak a significant amount of information. Don't use the ECB mode!

Look up "ECB Penguin" for a visual demonstration of how insecure ECB is.

Cipher block chaining, where the plaintext is XORd with the ciphertext of the previous block before encryption, can be show to be CPA-secure.

Output feedback, where the plaintext is XORd with the ciphertext after encryption, is also CPA-secure. In that case the bulk of the computation can be done independently of the message.

In counter mode, follow the steps in Figure **??**.

Not that kind of nonce.

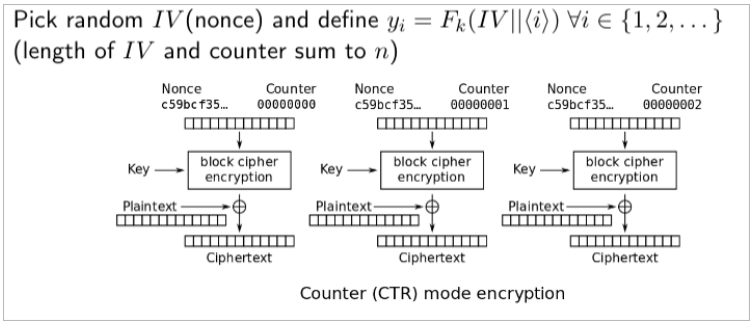


Figure 1: CTR Mode

Message Authenication Code

Encryption ensures confidentiality, but not integrity. A MITM might tamper with the ciphertext.

A *message authentication code* (MAC) consists of three PPT algorithms (`Gen`, `Mac`, `Verify`), such that:

- $\texttt{Gen}(1^n)$ outputs a key k
- $\texttt{Mac}_k(m)$ takes as input a key k and a message $m \in \{0,1\}^*$ and outputs a tag t.
- $\texttt{Verify}_k(m,t)$ takes as input a key k, a message m, and a tag t, and outputs 1 (accept) or 0 (reject)

A MAC allows a recipient to validate that a message was not tampered with and that it was sent by a key holder. We must require

- Correctness: if MAC comes from the authorized source, it should verify.
- Unforgeability: only authorized source can generate the MAC.

An attacker can resend previously authenticated message along with its (valid) tag. This can be precvented with a nonce like a counter or timestamp.

Authenticated Encryption

So far we have considered security and integrity seperately. Encryption thus far has not guaranteed integrity, and message authentication codes do not guarantee confidentiality. In fact, in many a scenario, lack of integrity implies lack of confidentiality.

This leads to the need for *authenticated encryption*, which incorporates a checksum. A simple way to do this is tag the message with a checksum and then encrypt both the message and tag (auth-then-enc). You may also send the MAC for the plaintext with the encrypted message (auth-and-enc). It turns out the only secure method is to encrypt and then compute the MAC for the encrypted message. This method, enc-then-auth, is secure. There are still several vulnerabilities:

- Re-ordering attack: swap order of messages
- Replay attack: replay a valid ciphertext previously sent
- Message dropping attack: drop some of the messages sent
- Reflection attack: take a ciphertext from Alice to Bob and send it back to Alice

To defeat these vulnerabilities, keep two counters $\texttt{ctr}_{A,B}$, $\texttt{ctr}_{B,C}$ and define a directionality bit b. A sends $c = \text{Enc}_k(b_{A,B}|\texttt{ctr}_{A,B}|m)$.

Hash Functions

A hash functions H has

- deterministic mapping from input to output
- input (long) binary strings
- output short fixed-length string (aka message digest)

A good hash function has as few collisions as possible. In data structures we aim to minimize collisions, but in cryptography we must avoid them. Since the range of a hash function is less than the domain (the output string is shorter than the input string), it's impossible to completely avoid collisions, but finding x_1, x_2 s.t. $H(x_1) = H(x_2)$ should be computationally infeasible.

We consider keyed hash functions with the difference that keys are public.

$$H^s(x) = H(s, x) \tag{6}$$

A hash function with output length $l(n)$ is a pair of PPT algorithms (`Gen`, `H`) s.t. `Gen` takes as input a security parameter 1^n and outputs a key s and `H` is a deterministic algorithm that takes as input a key s and a string $x \in \{0,1\}^*$ and outputs a string $H^s(x) \in \{0,1\}^{l(n)}$.

Collision resistance states that no PPT adversary can find a collision except with negligible probability. In practice, we use unkeyed hash functions with a fixed output length. Such hash functions are collision resistant for all practical purposes (colliding pairs are unknown and difficult to find).

For some applications it might make sense to use a weaker notion than collision resistance.

- Preimage resistance: Given s and $y = H^s(x)$ for a uniform x, it is infeasible for a PPT adversary to find a value x' (equal to x or not) such that $H^s(x') = y$. In other words, H^s is a one-way function.
- Second-preimage resistance: Given s and a uniform x it is infeasible for a PPT adversary to find $x' = x$ such that $H^s(x') = H^s(x)$.

A collision resistant hash function is also second-preimage resistant.

A second-preimage resistant hash function is also preimage resistant (under certain conditions).

Evaluating H with digest length l on $q \approx 1.2 \times 2^{\frac{l}{2}}$ yields a collision with probability $\frac{1}{2}$. For n-bit of effective security need digest length of at least $2n$.

Practical Constructions

We have seen private-key schemes constructed from pseudorandom generators, functions, and permutations. They can be constructed assuming one-way functions exist. This is a core principle in theoretical modern cryptography. A one-way function $f : \{0,1\}^* \rightarrow \{0,1\}^*$ is easy to compute, but hard to invert. The existence of one-way functions is still an open conjecture. Assuming one-way functions exist, we can create pseudorandom generators, functions, and permutations from them. This is of little practical significance currently, since these constructions are far less efficient than heuristic methods. In practice, we use heuristic constructions instead. Heuristic means they cannot be proven secure based on a weaker assumption. These constructions have gone through years of public scrutiny and cryptoanalysis.

Stream Ciphers

Remember our definition of a stream cipher as a pair of deterministic algorithms (`Init`, `Next`): `Init` takes input seed s; outputs initial state `st`. `Next` takes current state `st` and outputs a bit y and updated state `st'`. A stream cipher's output behaves like a PRG (indistinguishable from a uniform sequence). We can construct a heuristic stream cipher from Linear-Feedback Shift Registers (LFSR). However, these have a finite number of states and the output periodically repeats. This means that LFSRs are not by themselves secure stream ciphers. To make them secure, we need to add nonlinearity through any of a number of means.

Block Ciphers

Shannon introduced a paradigm for constructing random-looking permutations. Construct permutation F with large block length from smaller random (or random-looking) permutations $\{f_i\}$. The key for F will describe 16 permutations $f_1, ..., f1_6$ each with block length of 8 bits (1 byte). Given input $x \in \{0,1\}^{128}$, parse as 16 bytes $x_1...x_{16}$, then set $F_k(x) = f_1(x_1)||...||f_{16}(x_{16})$.

We say $\{f_i\}$ introduces confusion into F. However, F as described is not pseudorandom. If x and x' only differ in first bit, $F_k(x)$ and $F_k(x')$ will only differ in first byte. We add a diffusion step to permute (mix) the bits of the output. The goal is to spread a local change through the entire block. The confusion and diffusion steps are together called a round. Rounds are repeated multiple times (also suggested by Shannon).

Substitution-permutation networks (SPN) implement the confusion-diffusion paradigm. Permutations $\{f_i\}$ are fixed public substitution

functions, called an S-box: $f(x) = S(k \oplus x)$. An important property of an SPN is that a small change in input must affect every bit of the output. In SPN, this property translates to a single bit change to input of an S-box changes at least two bits in the output of the S-box, and mixing permutations ensures that bits output by any S-box affect the input to multiple S-boxes in the next round. S-boxes must be designed carefully.

Another approach to construct block ciphers: Feistel Networks. Each round has a round function F and round key k. Input to round is divided into two halves L and R. Output of the round is two values: $(R, L \oplus F_k(R))$. To invert, pass output as input to Feistel (with keys in reverse order). This works regardless of Fs, they don't even need to be invertible. Three and four round Feistel can be proven to be secure (assuming F is a pseudorandom function).

Hash Functions

Hash functions map larger input sequence to shorter "digest." The main security requirement is collision resistance. We want hash functions that can handle very long inputs. It is much easier to construct fixed-length hash functions (compression functions). We can extend a fixed-length compression function h into an arbitrary length input hash function H using the Merkle-Damgard Transform. We can prove that H is collision resistant if h is collision resistant. The Merkle-Damgard transform is used in MD5, SHA-1, and SHA-2. We use the Davies-Meyer construction to build compression functions from block ciphers. Encrypt the prev compression output from the chain using the message block as key, and XOR output. Under certain assumptions on the block cipher, the Davies-Meyer construction can be shown to be collision resistant, as long as l is sufficiently large.

The newest family, SHA-3, moves away from this construction. SHA-3 uses the sponge construction: data is "absorbed" into the sponge, and result is "squeezed" out.

Theory

Modulus

We say b is invertible modulo N if there exists an integer c such that $bc = 1 \mod N$. The multiplicative inverse of b modulo N is unique and usually denoted b^{-1}. When b is invertible, we define division as $[a/b \mod N] = [ab^{-1} \mod N]$.

With integers b, N where $b \geq 1$ and $N > 1$, b is invertible mod N iff $\gcd(b, N) = 1$.

Groups

A group is a set of elements $\mathcal{G}$ with a group operation $\odot$ on two elements, which operates on two elements and returns another element in the group.

$$\odot : \mathcal{G} \times \mathcal{G} \to \mathcal{G} \tag{7}$$

It has the properties of:

- Closure:
- Identity
- Inverse
- Associativity

Abelian groups also have the commutative property.

$\mathcal{Z}_N = \{0, \ldots, N-1\}$ for $N > 1$ is an abelian group with respect to addition modulo N.

The order of a group is the number of elements in that group: for the above example, the order is N.

We define $\mathcal{Z}_N^*$ as the set of integers mod N that are relatively prime to N: this is an abelian group under multiplication mod N.

Define $\Phi(N) = |\mathcal{Z}_N^*|$ as the order of the group $\mathcal{Z}_N^*$ ($\Phi(N)$ is Euler's totient formula). If $N = pq$ with p, q distinct primes, then $\Phi(N) = (p-1)(q-1)$.

Given a composite integer N, the factoring problem is to find integers $p, q > 1$ s.t. $pq = N$. Factoring is a classic hard problem. The smallest prime factor is at most $\sqrt{N}$. Consider the following experiment: choose two uniform n-bit integers x_1, x_2. Set $N = x_1 \times x_2$. Adversary is given N and guesses x_1', x_2'. Adversary wins if $N = x_1' \times x_2'$. This is a hard problem if x_1, x_2 are large primes. This is the basis of RSA.

Let $\mathcal{G}$ be a finite group of order m. Consider the set $< g >= \{g^0, g^1, \ldots\}$. We saw that $g^m = 1$. Ergo $< g >$ repeats after i terms $< g >= \{g^0, \ldots, g^{i-1}\}$. If there is an element $g \in \mathcal{G}$ with $i = m$,

then $< g >= \mathcal{G}$. We call $\mathcal{G}$ a cyclic group and g a generator of $\mathcal{G}$. This means that $\forall h \in \mathcal{G} \exists x \in \{0, \ldots, m-1\}$ s.t. $h = g^x$. For a cyclic group of order m with generator g we have $\mathcal{G} =< g >= \{g^0, g^1, \ldots, g^{m-1}\}$. We call x the discrete logarithm of h w.r.t. g and write $x = \log_g h$. For certain groups it's hard to compute the discrete logarithm. This is the basis of Diffie-Hellman problems, of which are two categories.

The CDH problem is to compute $DH_g(h1, h2) = g^{(\log_g h_1)(\log_g h_2)}$ from uniform $h_1 = g^{x_1}$ and $h_2 = g^{x_2}$.

The DDH problem is to distinguish $DH_g(h1, h2) = g^{(\log_g h_1)(\log_g h_2)}$ from a uniform group element.

It has been shown that the DDH problem is easy if the group order m has small prime factors. A popular option for Diffie-Hellman is to use cyclic group $\mathbb{Z}_p^*$ for prime p of form $p = 2q + 1$ where q is also prime (named after Sophie Germain).

Public Key Cryptography

We explored the use of private-key cryptography to help Alice and Bob communicate over an insecure channel. Assuming they both hold a shared secret key. They can meet up, or use a trusted third party to deliver the key. Might work in some cases (e.g. Cold War era "red phone"), but what if Alice needs to communicate with many more people? Establishing and storing so many private keys will be problematic.

Enter the Diffie-Hellman Key Exchange Protocol. In this scheme, they are both capable of generating a shared key based on private secret keys. It goes as follows:

- Alice and Bob agree on group $\mathbb{G}$ of order m and generator $g \in \mathbb{G}$ (in practice these parameters are standardized).
- Alice computes $k_A = P_B^a = (g^b)^a = g^{ab}$
- Bob computes $k_B = P_A^b b = (g^a)^b = g^{ab}$
- $k_A = k_B$

The discrete-logarithm problem guarantees that an adversary cannot compute the secret values of Alice and Bob (a, b) from messages communicated (P_A, P_B).

Diffie and Hellman also introduced the notion of public-key cryptography. Bob generates a key pair: public key to be disseminated, and secret key to be kept private. The public key serves as encryption key, and secret key serves as decryption key.

Public-key encryption is asymmetric. The receiver generates a pair of keys (pk, sk) and the sender uses the public key pk to encrypt, the receiever uses the private key sk to decrypt.

The channel must be authenticated or a man in the middle may substitute its own public keys and freely decrypt in the middle and re-encrypt going the other way.

Because the adversary has the public key, they may take any message and encrypt it. Thus any scheme we create should be CPA secure.

Public Key Encryption

One encryption scheme that can be derived from DH is El Gamal. The El Gamal Encryption Scheme is as follows:

- `Gen`: on Group $\mathbb{G}$ of order q and generator g, choose a uniform $x \in \mathbb{Z}_q$ and compute $h = g^x$. Public key is $(\mathbb{G}, q, g, h)$ and the private key is $(\mathbb{G}, q, g, x)$.
- `Enc`: on input public key $pk = (\mathbb{G}, q, g, h)$ and message $m \in \mathbb{G}$ choose uniform $y \in \mathbb{Z}_q$ and output ciphertext $g^y, h^y \cdot m$.
- `Dec`: on input private key $sk = (\mathbb{G}, q, g, x)$ and ciphertext (c_1, c_2) output $m = \frac{c_2}{c_1^x}$.

El Gamal encryption is, nicely, multiplicatively homomorphic.

An encryption scheme derived from RSA follows:

- `Gen`: run `GenRSA` to get (N, e, d), public key is $<N, e>$ and private key is $<N, d>$
- `Enc`: on input public key $pk = <N, e>$ and message $m \in \mathbb{Z}_N^*$, compute ciphertext $c = [m^e \mod N]$
- `Dec`: on input private key $sk = <N, d>$ and ciphertext $c \in \mathbb{Z}_N^*$ compute the message $m = [c^d \mod N]$

Digital Signatures

Providing privacy is independent from providing integrity. Digital signature schemes are public-key equivalent of MACs in private-key cryptography. A signer uses their private key sk to "sign" a message, and anyone with the singer's public key can verify the integrity of that signature.

Message authentication codes (MACs) require a shared secret key between parties. Digital signatures avoid the need to establish a secret key with every single receiver. The sender only computes a single signature that all recipients can verify. Digital signatures are publicly verifiable: everyone can check they got the same authenticated message. Digital signatures are also transferable: signatures can be shown to a third party and verified (important for certificates and public key infrastructure as we will see later). They also provide an important property of non-repudiation: if a sender signs a message, they cannot later deny doing that, which is not true for MACs.

A signature scheme consists of three PPT algorithms: `Gen`, `Sign`, `Verify` such that: Key generation algorithm `Gen` takes as input a security parameter 1^n and outputs a pair of keys (pk, sk). Signing algorithm `Sign` takes as input a private key sk and a message m and outputs a signature $\sigma = [m^d \mod N]$. Verification algorithm `Verify` takes as input a public key pk, a message m, and a signature σ. It outputs a bit b either a 1 meaning valid or 0 meaning invalid: $b = \texttt{Verify}_{pk}(m, \sigma)$.

An adversary should not be able to output a forgery even after seeing message/signature pairs.

Similar to encryption, signature schemes are less efficient than MACs. If we need to sign a long message, we can use a hybrid hash-and-sign construction where we generate a hash digest of the message and then sign the hash.

The RSA signature scheme can be constructed as follows:

- `Gen`: on input 1^n run `GenRSA` to obtain (N, e, d).
- `Sign`: on input a private key $sk = <N, d>$ and a message $m \in \mathbb{Z}_N^*$, compute signature $\sigma = [m^d \; mod N]$.
- `Verify`: on input a public key $pk = <N, e>$, a message m, and a signature $\sigma \in \mathbb{Z}_N^*$, output 1 iff $m = [\sigma^e \mod N]$.

An identification scheme is an interactive protocol that allows one party to prove its identity to another. We call the parties prover and verifier. Verifier has the public key, prover convinces the verifier that it holds the corresponding secret key.

The Schnorr Identification Scheme (Fiat-Shamir Transform) is one such scheme. The Digital Signature Algorithm (DSA) is also based on

the discrete-log problem. DSA follows a similar template as Schnorr by building an interactive identification scheme and collapsing it into a non-interactive signature scheme. Newer schemes include elliptic curve versions of DSA including ECDSA and EdDSA. Quantum secure signature schemes include ML-DSA and SLH-DSA.

Public Key Infrastructure

Public key cryptography is used in many applications:

- Secure web connections
- Software signing (against malware)
- Secure messaging and email
- Cryptocurrencies and blockchains

How do we know the public key of an entity? Through digital certificates!

A digital certificate is a signature binding an entity to a public key cert = Sign("Bob's key is pk_B"). Who will sign this? Once a single public key is securely transferred, we use it to trust other's keys. Specifying such details and other things (roles, policies, software, etc) constitute a public key infrastructure. The simplest PKI consists of a single certificate authority (CA). A certificate authority is trusted to issue certificates for everyone's public keys. Of course, it's ot very practical for everyone to trust the same CA. One CA'ss certification process may be insufficient. The CA is a single point of failure, but luckily we can rely on multiple CAs. Bob decides which CAs he wants to issue a certificate. Alice, presented with multiple certificates, decides which ones she trusts. The security is as good as the least-secure CA that Alice trusts. Another approach is through certificate chains.

A "root CA" can delegate issuing certificates to other parties. Each of these CAs can then issue certs for other parties.

What if someone's private key is leaked who previously got a certificate for their public key? We need to be able to generate a new private public pair and remove the old public key from circulation.

- Expiration: have certificates expire after a certain date:
- Revocation: have certificates include a serial number, and revoke certificates based on that.

To visit a website securely, each website needs to obtain a host certificate from a CA. The certificate includes info such as the domain name, person or organization it was issued for, issue date and expiration date of the certificate, and the public key. The user can verify that the website admin owns the website name (which prevents spoofing). The website encrypts the connection between user browser and itself (which prevents eavesdropping).

PGP uses hybrid encryption (symmetric-key + public-key). A symmetric key is generated by the sender. The symmetric key is used to encrypt the message, and is used only once (aka session key). The

session key is sent over using receiver's public key. Web of trust is a decentralized alternative to hierarchical CAs. Every user signs certificates for whoever they want.

System and Network Security Applications

Multi-Factor Authentication

Multi-Factor Authentication (MFA) is a familiar (and evocative) term for Purdue students.

We require authentication to log into a computer resource (network, device, application, etc.). User provides a piece of evidence (factor) for their identity: e.g., a password. Multi-factor (two-factor) authentication requires two or more pieces of such evidence, based on the premise that an attacker is unlikely to obtain both.

Factors can include physical objects (security token via a USB stick, card), something the user knows (password, PIN), something the user is (biometrics such as finger print or typing patterns), or location (within a network).

An algorithm is used to create a new sequence of characters valid for a single login session. The generated sequence of characters is essentially a one-time pad (OTP).

A secret key is shared between the client and server. Each passcode should be used only once, some variable to generate a different OTP each time user wants to log in.

- HOTP: HMAC-based one-time pass: variable is a counter.
- TOTP: Time-based one-time pass: variable is the current time.

TOTP is superior in almost every respect and is widely used in industry.

Universal Second Factor

Shared secret may be transmitted in plaintext between the client and server. Shared secret may be stored in plaintext on server. Passcode vulnerable to MITM attack. Passcode has to be entered manually, prone to error. Battery operated. Universal 2nd Factor (U2F) is an open standard that addresses these issues.

U2F device and client (e.g., web browser) are controlled by the user, and relying party is controlled by the server (doing authentication). U2F is based on a challenge-response protocol. U2F device holds a public key pair, and signs a challenge sent by the server during authentication.

Secret Sharing

Consider: s is the signature for a peace treaty. We need three parties – a general, a judge, and the president – to all agree to sign. Have each party receive shares s_1, s_2, s_3 of the secret s. Having one or two of the shares is not enough to get s If all three parties come together, they can obtain s.

We have a trusted secret broker compute s_1 and s_2 randomly, and choose s_3 s.t. $s_1 \oplus s_2 \oplus s_3 = s$. Out of 3 shares, you need 3 to recover the secret. This can be generalized to n-out-of-n secret sharing.

In 1979, Adi Shamir introduced a t-out-of-n secret sharing scheme where any t shares (but no fewer) could reconstruct the shared secret. Shamir reasoned that t points are required to specify a $t-1$-degree polynomial. However, with $t-1$ points, there are infinitely many polynomials which fit the points. If the secret is, for instance, the coefficients of the polynomial, or the value of the y intercept. For the latter, it suffices to sample the polynomial at random points to generate each of the secret shares.

One t shares are obtained, we can use Lagrange interpolation to recover the polynomial. Any subset of up to $t-1$ shares does not leak any information about the secret. This scheme is perfectly secure and very flexible! For instance, One can increase n and add new shares without affecting other shares. Existing shares can be removed without affecting other shares. It is possible to replace all shares (or even change t) without changing the secret and without revealing any information on the secret by selecting a new polynomial $\hat{P}(x)$ and a new set of shares. Some parties can be given more than one share.

Threshold

Threshold Encryption

The secret key scheme proposed previously has two issues, namely:

- The dealer knows the secret key
- The secret key must be assembled to be used

We construct a t-out-of-n threshold cryptosystem for n parties $P_1, ..., P_n$ consisting of three algorithms.

- Distributed key generation: A protocol between parties for generating a public pk key such that party P_i obtains a private share s_i of the private key sk corresponding to public key pk (algorithm depends on t).
- Encryption: An algorithm that takes as input plaintext message m, public key pk, and outputs ciphertext c of m under public key pk.
- Decryption: A protocol between any t parties $P_{i_1}, \ldots, P_{i_t}$ that on input c, private shares $s_{i_1}, /dots, s_{i_t}$, outputs plaintext m.

For this scheme, the private key sk remains secret at all times unless t or more parties cheat, and no trusted dealer is required. Threshold El Gamal Encryption meets this scheme. Additionally, Threshold El Gamal Encryption is multaplicatively homomorphic,

Multi-party Computation

Say we have two secret values x and y which are secret shared using Shamir's secret sharing scheme. We want to compute $z = x + y$ without revealing either value. Each party can compute $z_i = x_i + y_i$ and t of these shares are sufficient to compute z, without revealing information about x or y (because only z_i is shared with other parties and not x_i or y_i).

Interestingly, while addition can be performed locally, multiplication requires communication. Since the degree of the polynomial becomes $t - 1 + t - 1 = 2t - 2$ and thus we need $2t - 1$ shares to reconstruct the polynomial obtained by multiplying x and y. Since we previously specified that no more than $t - 1$ parties would collaborate, we cannot perform multiplication without breaking the assumptions. However, parties may engage in a polynomial degree reduction protocol to reduce z to degree $t - 1$, which can then be recovered. It's pretty common behavior for one operation to be feasible and another to require collaboration, hence fully homomorphic encryption is a difficult problem.

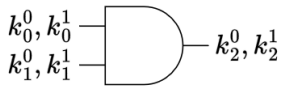
Oblivious Transfer

The concept behind oblivious transfer is a protocol between Alice and Bob, where Alice has two inputs and Bob inputs either a 0 or 1, indicating which of the inputs from Alice Bob would like to know. Alice should learn nothing about Bob's choice, and Bob should learn nothing about the input he did not select.

Yao's Garbled Circuits

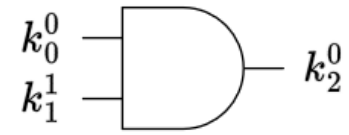
In oblivious transfer, the function f can be transformed into a Boolean circuit (possible for all polynomial time functions) and represented as a directed acyclic graph using only AND and XOR gates for the nodes and boolean valued wires for the edges.

Assign one encryption key for every possible boolean value on wires. "Double encrypt" output keys with corresponding input keys.



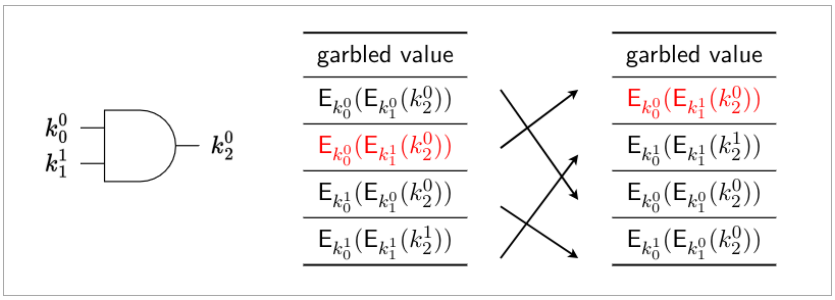
w_0	w_1	w_2	garbled value
k_0^0	k_1^0	k_2^0	$\mathsf{E}_{k_0^0}(\mathsf{E}_{k_1^0}(k_2^0))$
k_0^0	k_1^1	k_2^0	$\mathsf{E}_{k_0^0}(\mathsf{E}_{k_1^1}(k_2^0))$
k_0^1	k_1^0	k_2^0	$\mathsf{E}_{k_0^1}(\mathsf{E}_{k_1^0}(k_2^0))$
k_0^1	k_1^1	k_2^1	$\mathsf{E}_{k_0^1}(\mathsf{E}_{k_1^1}(k_2^1))$

A garbled gate consists of the doubly encrypted output key values.



garbled value
$\mathsf{E}_{k_0^0}(\mathsf{E}_{k_1^0}(k_2^0))$
$\mathsf{E}_{k_0^0}(\mathsf{E}_{k_1^1}(k_2^0))$
$\mathsf{E}_{k_0^1}(\mathsf{E}_{k_1^0}(k_2^0))$
$\mathsf{E}_{k_0^1}(\mathsf{E}_{k_1^1}(k_2^1))$

How do we evaluate a garbled gate? Using keys corresponding to input values: k_0^0, k_1^1 (corresponding to 01 input). Note that key values don't reveal anything about the actual wire Boolean value. We can give input keys to a party and ask them to evaluate the gate; but they can obtain the output key without learning the wire values on input/output. We need to randomly permute rows (and use encryption with wrong key detection).


k_0^0
k_1^1
k_2^0
garbled value
$\mathsf{E}_{k_0^0}(\mathsf{E}_{k_1^0}(k_2^0))$
$\mathsf{E}_{k_0^0}(\mathsf{E}_{k_1^1}(k_2^0))$
$\mathsf{E}_{k_0^1}(\mathsf{E}_{k_1^0}(k_2^0))$
$\mathsf{E}_{k_0^1}(\mathsf{E}_{k_1^1}(k_2^1))$
garbled value
$\mathsf{E}_{k_0^0}(\mathsf{E}_{k_1^1}(k_2^0))$
$\mathsf{E}_{k_0^1}(\mathsf{E}_{k_1^1}(k_2^1))$
$\mathsf{E}_{k_0^0}(\mathsf{E}_{k_1^0}(k_2^0))$
$\mathsf{E}_{k_0^1}(\mathsf{E}_{k_1^0}(k_2^0))$